Common collections

The collections we're inspecting in this chapter are stored on the heap; We'll discuss three collections that are used very often in Rust programs:

- A 'vector' allows you to store a variable number of values next to each other.
- A 'string' is a collection of characters.
- A 'hash map' allows you to associate a value with a particular key.
  It's a particular implementation of the more general data structure called a map.

Vectors: storing list of values

ectors allow you to store more than one value in a single data structure that puts all the values next to each other in memory.
Vectors can only store values of the same type.

They are useful when you have a list of items, such as the lines of text in a file or the prices of items in a shopping cart.

Creating a new Vector

To create a new, empty vector, we can call the 'Vec::new()' function, as shown:

```
let v: Vec<i32> = Vec::new();
```

Note that we added a type annotation here. Because we aren't inserting any values into this vector, Rust doesn't know what kind of elements we intend to store.

The `Vec<T>` type provided by the standard library can hold any type, and when a specific vector holds a specific type, the type is specified within angle brackets: this is called 'generics' and for the previous example we've told rust that 'v' will hold i32 elements.

Realistically Rust can often infer the type of value you want to store once you insert values, so you rarely need to do this type annotation.

```
let v = vec![1, 2, 3];
```

Updating a Vector

Obviously vectors provide the 'push()' method which allows us to insert elements inside them.
As for variables, if we want to manipulate the Vector it has to be declared as mutable.

```
let mut v = Vec::new();

v.push(5);
v.push(6);
v.push(7);
v.push(8);
```

Dropping a Vector drops its elements

Like any other struct, a vector is freed when it goes out of scope:

```
{
    let v = vec![1, 2, 3, 4];

    // do stuff with v
} // <- v goes out of scope and is freed here
```

When the vector gets dropped, all of its contents are also dropped, meaning those integers it holds will be cleaned up.

Reading elements of Vectors

There are two ways to reference a value stored in a Vector: either with indexing syntax or the get method

```
let v = vec![1, 2, 3, 4, 5];

let third: &i32 = &v[2];
let third: Option<&i32> = v.get(2);
```

Note that using the second method will return us a 'Option';

When we run this code, the first '[]' method will cause the program to panic because it references a nonexistent element.
This method is best used when you want your program to crash if there's an attempt to access an element past the end of the vector.

When the get method is passed an index that is outside the vector, it returns None without panicking. You would use this method if accessing an element beyond the range of the vector happens occasionally under nor- mal circumstances.
Your code will then have logic to handle having either 'Some(&element)' or 'None', as discussed earlier.

Recall the rule that states you can't have mutable and immutable ref- erences in the same scope, for example if we hold an immutable reference to the first element in a vector and try to add an element to the end, which won't work:

```
let mut v = vec![1, 2, 3, 4, 5];

let first = &v[0];

v.push(6);
```

```
                        error[E0502]: cannot borrow 'v' as mutable because it is also borrowed as
immutable
```

This error is due to the way vectors work: adding a new element onto the end of the vector might require allocating new memory and copying the old elements to the new space, if there isn't enough room to put all the elements next to each other where the vector currently is.
In that case, the reference to the first element would be pointing to deallo- cated memory.

Iterating over values in a Vector

We can use a 'for' loop to iterate over a Vector to get immutable references to each element:

```
let v = vec![100, 32, 57];
```

```
for i in &v {
    println!("{}", i);
}
```

We can also iterate over mutable references to each element in a mutable vector in order to make changes to all the elements.
In the following example we are adding 50 to each element while iterating over the Vector:

```
let mut v = vec![100, 32, 57];

for i in &mut v {
    *i += 50;
}
```

To change the value that the mutable reference refers to, we have to use the dereference operator '*' to get to the value in 'i' before we can use the += operator.

Using Enum to store multiples Types

Earlier in this chapter we mentioned that a Vector can only store one Type at a time, but it isn't quite the case: the variants of an Enum are defined under the same Enum Type, so when we need to store elements of a different Type in a Vector, we can define and use an Enum.

Say we want to get values from a row in a spreadsheet in which some of the columns in the row contain integers, some floating- point numbers, and some strings.
We can define an Enum whose variants will hold the different value Types, and then all the Enum variants will be considered the same type: that of the enum.
Then we can create a Vector that holds that Enum and so, ultimately, holds different types.

```
enum SpreadsheetCell {
    Int(i32),
    Float(f64),
    Text(String),
}

let row = vec![
    SpreadsheetCell::Int(3),
    SpreadsheetCell::Text(String::from("blue")),
    SpreadsheetCell::Float(10.12),
];
```

Using an Enum plus a 'match' expression means that Rust will ensure at compile time that every possible case is handled.

When you're writing a program, if you don't know the exhaustive set of types the program will get at runtime to store in a Vector, the Enum technique won't work;

So we can use a Trait object that we will discuss later on.

Strings

On a deeper level the only representation Rust has about a string is from the 'str' atomic type;

The 'String' type, which is provided by Rust's standard library is a growable, mutable, owned, UTF-8 encoded string type.
Rust's standard library also includes a number of other string types, such as 'OsString', 'OsStr', 'CString', and 'CStr', they refer to owned and borrowed variants, just like the 'String' and 'str' types

they refer to owned and borrowed variants, just like the 'String' and 'str' types.

Creating a New String

Many of the same operations available with `Vec<T>` are available with 'String' as well, starting with the 'new()' function to create a String.

```
let mut s = String::new();
```

Often, we'll have some initial data that we want to start the string with;
for that, we use the 'to_string()' method, which is available on any type that implements the 'Display' trait as string literals:

```
let data = "initial contents";

let s = data.to_string();

// the method also works on a literal directly:
let s = "initial contents".to_string();
```

We can also use the function 'String::from()' to create a String from a string literal:

```
let s = String::from("initial contents");
```

Updating a String

You can conveniently use the '+' operator or the 'format!' macro to concatenate String values.

We can grow a String by using the push_str method to append a string slice:

```
let mut s = String::from("foo");
s.push_str("bar");
```

After these two lines, 's' will contain foobar.
The 'push_str()' method takes a string slice because we don't necessarily want to take ownership of the parameter.

The 'push()' method takes a single character as a parameter and adds it to the String.

```
let mut s = String::from("lo");
s.push('l');
```

this adds the letter l to a String using the 'push()' method.

Often, you'll want to combine two existing strings. One way is to use the '+' operator, as shown:

```
let s1 = String::from("Hello, ");
let s2 = String::from("world!");
let s3 = s1 + &s2; // note s1 has been moved here and can no longer be used
```

The reason 's1' is no longer valid after the addition and the reason we used a reference to 's2' has to do with the signature of the method that gets called when we use the + operator which is:

```
fn add(self, s: &str) -> String {
```

The reason we're able to use '&s2' in the call to add is that the compiler can coerce the '&String' argument into a '&str'.
When we call the add method, Rust uses a deref coercion, which here turns '&s2' into '&s2[…]'

Because add does not take ownership of the 's' parameter, 's2' will still be a valid String after this operation.

add takes ownership of 'self', because 'self' does not have an '&'. This means 's1' will be moved into the add call and no longer be valid after that.
So although let `s3 = s1 + &s2` ; looks like it will copy both strings and create a new one, this statement actually takes ownership of 's1', appends a copy of the contents of 's2', and then returns ownership of the result.

If we need to concatenate multiple strings:

```
let s1 = String::from("tic");
let s2 = String::from("tac");
let s3 = String::from("toe");

let s = s1 + "-" + &s2 + "-" + &s3;
// s will contain 'tic-tac-toes'
```

For more complicated string combining, we can use the 'format!' macro:

```
let s1 = String::from("tic");
let s2 = String::from("tac");
let s3 = String::from("toe");

let s = format!("{}-{}-{}", s1, s2, s3);
// s will contain 'tic-tac-toes'
```

Instead of printing the output to the screen, it returns a String with the contents.
The version of the code using 'format!' is much easier to read and doesn't take ownership of any of its parameters.

Indexing into Strings

A String is a wrapper over a `Vec<u8>` .

```
let len = String::from("Hola").len();
```

A reason Rust doesn't allow us to index into a String to get a character is that indexing operations are expected to always take constant time (O(1)). But it isn't possible to guarantee that performance with a String, because Rust would have to walk through the contents from the beginning to the index to determine how many valid characters there were.

Slicing Strings

Indexing into a string is often a bad idea because it's not clear what the return type of the string-indexing operation should be: therefore, Rust asks you to be more specific if you really need to use indices to create string slices.
To be more specific in your indexing and indicate that you want a string slice, rather than indexing using '[]' with a single number, you can use '[]' with a range to create a string slice containing particular bytes:

```
let hello = "Здравствуйте";

let s = &hello[0..4];
```

Here, ‘s’ will be a ‘&str’ that contains the first 4 bytes of the string.
Earlier, we mentioned that each of these characters was 2 bytes, which means ‘s’ will be Зд. What if we used &hello[0…1]?
Rust would panic at runtime in the same way as if an invalid index were accessed in a Vector;

So, please be VERY CAREFUL when creating the range for string slices, because they can crash your program!

Methods for iterating over a String

If you need to perform operations on individual Unicode scalar values, the best way to do so is to use the ‘chars()’ method.

```
for c in "larry".chars() {
    println!("{}", c);
}
```

This snippet will print:

```
l
a
r
r
y
```

The ‘bytes()’ method returns each raw byte.

Hash Maps: storing Keys with associated Values

The type `HashMap<K, V>` stores a mapping of keys of type K to values of type V.
It does this via a *hashing function* which determines how it places these keys and values into memory.

Hash maps are useful when you want to look up data not by using an index, as you can do with vectors, but by using a key that can be of any type.
For example in a game, you can keep track of each team’s score in a hash map in which each key is a team’s name and the values are each team’s score: given a team name, you can retrieve it’s score.

Let’s dive into the basic API of hash maps, keeping in mind that more useful ones are available in the Standard Library:

Creating a new Hash Map

You can create an empty HashMap with `new` and add elements with `insert`.

In the following example we’re keeping track of the scores of two teams whose names are Blue and Yellow, Blue starts with 10 points and Yellow with 50.

```
use std::collections::HashMap;

let mut scores = HashMap::new();

scores.insert(String::from("Blue"), 10);
scores.insert(String::from("Yellow"), 50);
```

As you can see the ‘HashMap data structure’ is indeed into ‘std’ but inside the collection portion of the Standard Library.

Just like vectors, HashMaps store their data on the heap.

This specific HashMap we've just created has keys of type `string` and values of type `i32`, and they're homogenous too so those must be the only two supported types for the whole liftime of this specific HashMap.

Another way of constructing a hash map is by using the collect method on a vector of tuples, where each tuple consists of a key and its value. Thecollect method gathers data into a number of collection types, including HashMap.

For example, if we had the team names and initial scores in two separate vectors, we could use the zip method to create a vector of tuples where "Blue" is paired with 10, and so forth. Then we could use the collect method to turn that vector of tuples into a hash map.

```
use std::collections::HashMap;

let teams = vec![String::from("Blue"), String::from("Yellow")];
let initial_scores = vec![10, 50];

let scores: HashMap<_, _> = teams.iter().zip(initial_scores.iter()).collect();
```

The type annotation `HashMap<_, _>` is needed here because it's possible to collect into many different data structures and Rust doesn't know which you want unless you specify. For the parameters for the key and value types, however, we use underscores, and Rust can infer the types that the hash map contains based on the types of the data in the vectors.

Hash Maps and Ownership

For types that implement the `Copy` trait such as `i32` the values are copied into the HashMap. For owned values such as `String` the values will be moved and the hash map will be the owner of those values, as follows:

```
use std::collections::HashMap;

let field_name = String::from("Favorite color");
let field_value = String::from("Blue");

let mut map = HashMap::new();
map.insert(field_name, field_value);

// field_name and field_value are invalid at this point!
```

If we insert references to values into the hash map, the values won't be moved into the hash map.
The values that the references point to must be valid for at least as long as the hash map is valid.

Accessing values in a Hash Map

We can get a value out of the hash map by providing its key to the get method:

```
use std::collections::HashMap;

let mut scores = HashMap::new();

scores.insert(String::from("Blue"), 10);
scores.insert(String::from("Yellow"), 50);

let team_name = String::from("Blue");
let score = scores.get(&team_name);
```

Here, `score` will have the value that's associated with the Blue team, and the result will be `Some(&10)`. The result is wrapped in `Some` because get returns an `Option<&V>`;
if there's no value for that key in the hash map, get will return `None`.

We can iterate over each key/value pair in a hash map in a similar man- ner as we do with vectors, using a for loop:

```
use std::collections::HashMap;

let mut scores = HashMap::new();

scores.insert(String::from("Blue"), 10);
scores.insert(String::from("Yellow"), 50);

for (key, value) in &scores {
    println!("{}: {}", key, value);
}

// Prints, in arbitrary order:
// Yellow: 50
// Blue: 10
```

Updating a Hash Map

Although the number of keys and values is growable, each key can only have one value associated with it at a time. When you want to change the data in a hash map, you have to decide how to handle the case when a key already has a value assigned: this is what we're going to look at now.

Overwriting a value

If we insert a key and a value into a hash map and then insert that same key with a different value, the value associated with that key will be replaced.

```
use std::collections::HashMap;

let mut scores = HashMap::new();

scores.insert(String::from("Blue"), 10);
scores.insert(String::from("Blue"), 25);

println!("{:?}", scores);

// Prints: {"Blue": 25}
```

The original value of 10 has been overwritten.

Only inserting a value if the key has no value

It's common to check whether a particular key has a value and, if it doesn't, insert a value for it.
Hash maps have a special API for this called entry that takes the key you want to check as a parameter. The return value of the entry method is an enum called `Entry` that represents a value that might or might not exist.

Let's say we want to check whether the key for the Yellow team has a value associated with it. If it doesn't, we want to insert the value 50, and the same for the Blue team.

Using the `Entry` API:

```
use std::collections::HashMap;
```

```
use std::collections::HashMap;

let mut scores = HashMap::new();
scores.insert(String::from("Blue"), 10);

scores.entry(String::from("Yellow")).or_insert(50);
scores.entry(String::from("Blue")).or_insert(50);

println!("{:?}", scores);

// Prints: "Yellow": 50, "Blue": 10
```

The `or_insert` method on `Entry` is defined to return a mutable reference to the value for the corresponding `Entry` key if that key exists, and if not, inserts the parameter as the new value for this key and returns a mutable reference to the new value.

Updating a value based on the old value

Another common use case for hash maps is to look up a key's value and then update it based on the old value.

The following snippet counts how many times each word appears in some text.
We use a hash map with the words as keys and increment the value to keep track of how many times we've seen that word.
If it's the first time we've seen a word, we'll first insert the value 0.

```
use std::collections::HashMap;

let text = "hello world wonderful world";

let mut map = HashMap::new();

for word in text.split_whitespace() {
    let count = map.entry(word).or_insert(0);
    *count += 1;
}
println!("{:?}", map);

// Prints: "world": 2, "hello": 1, "wonderful": 1
```

Hashing functions

If you profile your code and find that the default hash function is too slow for your purposes, you can switch to another function by specifying a different hasher.

A hasher is a type that implements the `BuildHasher` trait.